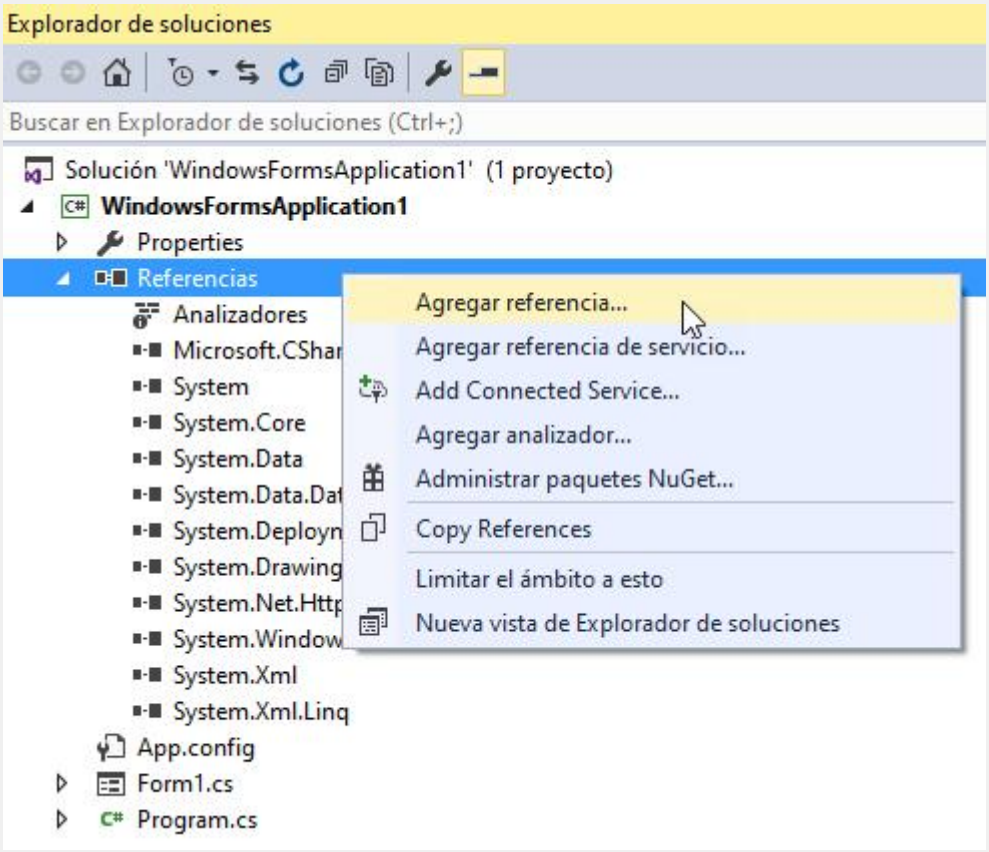
Clase LN (Library New)

La clase LN ha sido concebida como una biblioteca de funciones (métodos) que dan apoyo al programador de C# y que ahorrarían gran cantidad de líneas de programación. Por otra parte, disminuye en gran medida la pérdida de tiempo producida por el **"cómo se hace"** en tiempo de programación. Este documento esta básicamente orientado al programador neófito de C# y por tal razón se trata de ser lo más explícito posible en el documento.

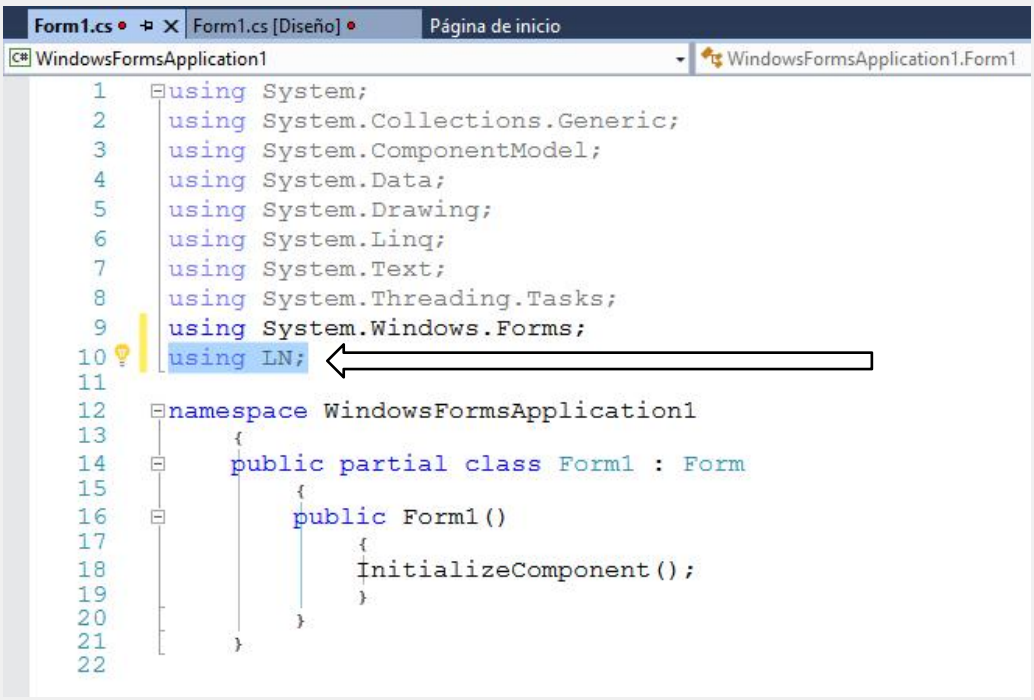
Dicha clase viene representada por la biblioteca dinámica LN.DLL adjunta a este proyecto.

Para poder trabajar con los métodos de la clase LN se deben seguir los siguientes pasos:

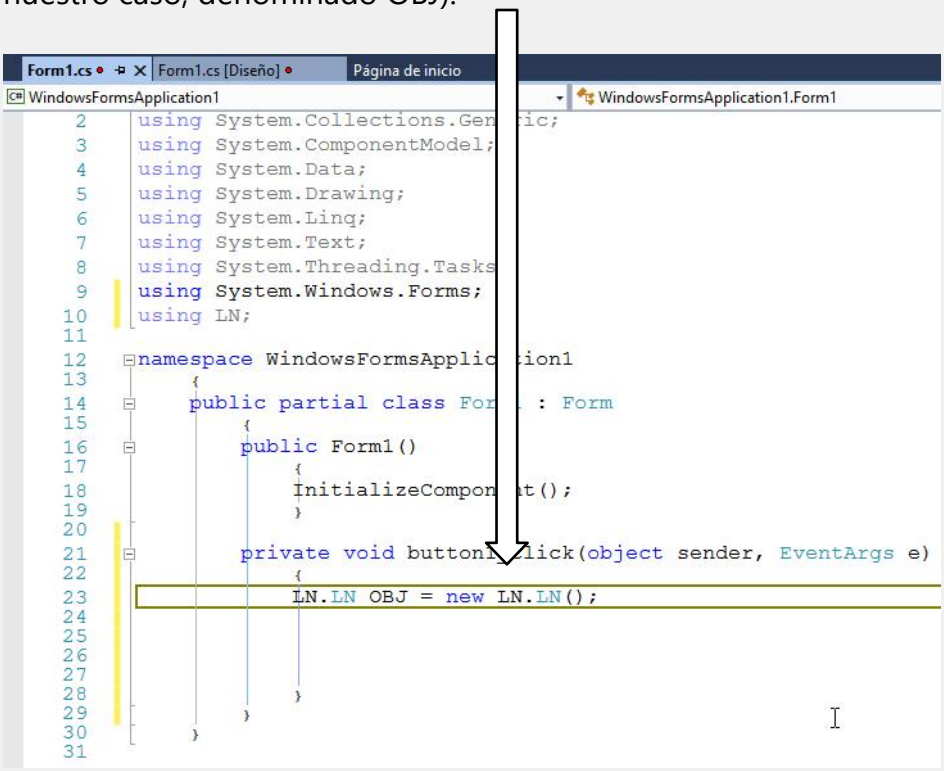
1) Comenzamos agregando una referencia a dicha DLL, de la forma siguiente:




2) Seguidamente agregamos el espacio de nombres LN mediante el **using** correspondiente dentro del código de programa en la forma tradicional:




3) A partir de este momento ya se puede instanciar el objeto con el nombre deseado (en nuestro caso, denominado OBJ):




4) Ahora al escribir **OBJ** seguido de un punto rápidamente le aparecerá la lista completa de todos los métodos de dicha clase, y bastará con seleccionar el método deseado.

La clase LN está concebida fundamentalmente para aplicaciones de tipo **Winforms (.exe)**, pero en el documento se especifica claramente cuales métodos pueden ser utilizados en aplicaciones **WEB** y cuales en aplicaciones **Winforms**.

**METODOS DE LA CLASE LN**


**MÉTODOS DE EMISION DE MENSAJES ..... 6**
**public void MensINFO(string Mensaje) ..... 6**
**public void MensERROR(string Mensaje) ..... 6**
**public bool MensConfirm(string Mensaje) ..... 6**
**public void MsgTemp(string M, int Seg) ..... 7**
**MailSender y SendMail ..... 8**
**public void MailSender ..... 9**
**public void SendMail(MailMessage MessageParameters) ..... 11**
**MÉTODOS DE TRABAJO CON FECHAS ..... 11**
**public int CheckDateUS(string Fecha) ..... 11**
**public int CheckDateESP(string Fecha) ..... 12**
**public string NormalizeDateUS(string FechaEntrada) ..... 13**
**public string NormalizeDateESP_Str(string FechaEntrada) ..... 13**
**public string NormalizeDateISO_Str(string FechaEntrada) ..... 14**
**public string ExtractHour_STR_HM(DateTime Fecha) ..... 14**
**public string NormalizeHour_STR(string HoraEntrada) ..... 15**
**public string ExtractDateUS_Str(DateTime Fecha) ..... 15**
**public string ExtractDateESP_Str(DateTime Fecha) ..... 15**
**public string ConvertDateUStoESP_Str(string FechaEntrada) ..... 15**
**public string ConvertDateESPtoUS_Str(string FechaEntrada) ..... 16**
**public string ConvertDateISOtoUS_Str(string FechaEntrada) ..... 16**
**public string ConvertDateISOtoESP_Str(string FechaEntrada) ..... 17**
**public int EdadIntESP(string FNac) ..... 17**
**public double EdadDecESP(string FNac) ..... 18**
**public int EdadIntUS(string FNac) ..... 19**
**public int EdadDecUS(string FNac) ..... 20**
**public int EdadDesdeHastaESP(string Desde, string Hasta) ..... 20**
**public int EdadDesdeHastaUS(string Desde, string Hasta) ..... 21**
**public double EdadDesdeHastaDecESP(string Desde, string Hasta) ..... 22**
**public double EdadDesdeHastaDecUS(string Desde, string Hasta) ..... 23**
**public double DateDifD(DateTime DESDE, DateTime HASTA) ..... 23**
**public double DateDifH(DateTime DESDE, DateTime HASTA) ..... 24**
**public double DateDifM(DateTime DESDE, DateTime HASTA) ..... 25**
**public bool NormalizeDate(int A, int M, int D, ref int ANorm, ref int MNorm, ref int DNorm) ..... 25**
**MÉTODOS DE VALIDACIÓN DE DATOS ..... 27**
**public bool SSNCheck(string SSN) ..... 27**
**public bool NUM(string Numero) ..... 27**
**public bool NUMERIC(string Numero) ..... 28**

public bool ZERO(string Numero) ..... 28
public bool ALFANUM(string Cadena) ..... 28
public bool ALF(string Cadena) ..... 29
public bool ALFEng(string Cadena) ..... 29
public bool Bisiesto(string AAAAStr) ..... 30
public bool BetweenInc(int N, int a, int b) ..... 30
public bool BetweenExc(int N, int a, int b) ..... 30
public bool BetweenIncDec(double N, double a, double b) ..... 31
public bool BetweenExcDec(double N, double a, double b) ..... 31
bool VINCheck(string VIN) ..... 32
MÉTODOS DE EJECUCIÓN DE PROCESOS ..... 32
public int MSDOSWait(string comando) ..... 32
public int MSDOSNoWait(string comando) ..... 33
public void PAUSE() ..... 33
public int ExecProcess(string comando) ..... 33
public int ExecProcessNoWait(string comando) ..... 34
MÉTODOS DE FUNCIONES DE CADENA DE CARACTERES ..... 35
public int FindStr(string Cadena, string Subcadena) ..... 35
public string FillString(string Car, int Nveces) ..... 36
public string DeleteChar(string Cadena, string CHAR) ..... 36
public string DeleteStr(string Cadena, string PATRON) ..... 37
public void SplitString(ref string[ ] Resultado, string Cadena, string[ ] delimitador) . 38
METODOS GENERALES ..... 39
public double Porciento(double P, double N) ..... 39
public string Trunc(double Origen, int Enteros, int Decimales) ..... 40
public void Upper(ref string Cadena) ..... 40
public void Lower(ref string Cadena) ..... 41
public void ShowRPT(string PathReport) ..... 41
Método FileDiskOpenCreate ..... 42
Método FileDiskOpenAppend ..... 43
Método FileDiskOpenRead ..... 44
Método FileDiskWrite ..... 45
Método FileDiskRead ..... 46
Método FileDiskCloseWriter ..... 47
Método FileDiskCloseReader ..... 48
Método GetDirectoryFilesDialog ..... 49
Método GetDirectoryFilesPath ..... 50
Método GetDirectoryFoldersDialog ..... 51
Método GetDirectoryFoldersPath ..... 52
Método CreateFileParameter ..... 54
Método CreateFileParameterArray ..... 55
Método StorageSize ..... 56

**MÉTODOS DE EMISION DE MENSAJES**

**public void MensINFO(string Mensaje)**

**Uso: Winforms App**
**Descripción:**
Emite un mensaje informativo con el texto que aparece en el parámetro de entrada.

**Parámetros de Entrada:**
string Mensaje

**Retorno:**
No Tiene

**public void MensERROR(string Mensaje)**

**Uso: Winforms App**
**Descripción:**
Emite un mensaje de error con el texto que aparece
en el parámetro de entrada.

**Parámetros de Entrada:**
Mensaje

**Retorno:**
No Tiene

**public bool MensConfirm(string Mensaje)**

**Uso: Winforms App**
**Descripción:**
Emite un mensaje de petición de confirmación cuyo texto se encuentra
en el parámetro de entrada.

**Parámetros de Entrada:**
string Mensaje

**Retorno:**
**True –** Aceptado
**False -** Denegado

**public void MsgTemp(string M, int Seg)**

**Uso: Winforms App**
**PARAMETROS:**
string M
int Seg

**RETORNO:**
NO TIENE

**DESCRIPCION:**

El método MsgTemp muestra un mensaje temporal de modo tal que al cumplirse el tiempo transcurrido, dicho mensaje desaparece y continua la ejecución del programa.

El parámetro de entrada M de tipo string, contiene el mensaje a mostrar, mientras que el segundo parámetro Seg especifica el número de segundos que permanecerá el mensaje en pantalla.

La longitud máxima del mensaje es de 60 caracteres, caso contrario se produce un mensaje de error.

Ejemplo:

```
LN.LN Objeto = new LN.LN();

...

Objeto.MsgTemp("Espere un momento por favor...", 3);
```

* Este mensaje dura solamente 3 segundos antes de desaparecer

**MailSender y SendMail**

**Uso: Winforms App & WEB App**

Los métodos **MailSender**, y **SendMail** que se explicarán a continuación están concebidos para poder automatizar el envío de un **e-mail** a uno o más destinatarios desde un programa escrito en C#, sin embargo, es necesario aclarar la advertencia de **GMAIL** al respecto, la cual se muestra a continuación:

==========

Es necesario permitir el acceso a aplicaciones no seguras desde la configuración de Gmail tal como se describe en la documentación de Gmail.

Si no lo habilitas recibirás una alerta de seguridad por parte de Google indicando que una aplicación diferente a Gmail ha accedido a tu correo y la aplicación te mostrara este mensaje.

The SMTP server requires a secure connection or the client was not authenticated. The server response was: 5.7.0 Authentication Required.

***Advertencia*** *El equipo de .NET ha marcado la clase SmtpClient como obsoleta. En su lugar recomiendan* ***MailKit***

**Configuración de Gmail**

La configuración necesaria para el cliente de SMTP de acuerdo a la documentación de Gmail es la siguiente:

| Parámetro | Valor |
|---|---|
| Servidor de correo saliente (SMTP) | smtp.gmail.com |
| Requiere TLS | Sí (si está disponible) |
| Requiere autenticación | Sí |
| Puerto para TLS/STARTTLS | 587 |

| | |
|---|---|
| Nombre completo o nombre mostrado | Tu nombre |
| Nombre de la cuenta, nombre de usuario o dirección de correo electrónico | Tu dirección de correo electrónico completa |
| Contraseña | Tu contraseña de Gmail |

Para usar estos métodos en otros proveedores como Yahoo, Hotmail u otro cualquiera, se deberá obtener información de los mismos, caso que se requiera.

**public void MailSender**

**Uso: Winforms App & WEB App**

**MailSender** es un método declarativo establece parámetros del mail en forma declarativa.

**PARAMETROS:**

```
(string SMTP,
 int    PORT,
 string Login,
 string Password,
 string SUBJECT,
 string FROM,
 string[] TO,
 string ATTACH,
 string[] MessageBody)
```

**DESCRIPCION:**

El método MailSender es el método principal de mensajería de envíos de correo desde programa y el mismo establece todos los parámetros necesarios para que el e-mail se transmita correctamente.

Los parámetros son los siguientes:

**NOTA IMPORTANTE PARA CORREO GMAIL** (**Extraído De GOOGLE**):

El servicio SMTP de Gmail es ofrecido con limitaciones de envíos para evitar que los spammers utilicen su servicio para enviar correos basura.

Los límites establecen 100 destinatarios a la vez, y 500 mensajes por día: si usted supera esta restricción, Google bloqueará su cuenta.

Además, su servicio SMTP utiliza IP's compartidas que no garantizan la entrega, ya que estas también son utilizadas por spammers que dañan seriamente la reputación de estas direcciones, incrementando las probabilidades de que su email legítimo sea filtrado por sistemas antispam por error.

Por esta razón, si usted está enviando una campaña de email o necesita enviar emails ilimitados, la mejor alternativa consiste en optar por un servidor SMTP profesional contratado al proveedor de servicios y por un precio determinado usted podrá realizar envíos sin restricciones a los usuarios que desee.

PARAMETROS DE ENTRADA:
**----------**
**string SMTP,**
**----------**
Aquí se declara la dirección del servidor de envío (SMTP) que corresponde al cliente de mensajería que se desea utilizar.

**Por ejemplo:**
**"smtp.gmail.com"** para correo GMAIL
**"smtp.mail.yahoo.com"** para correo YAHOO
**"smtp.live.com"** para correo HOTMAIL
**----------**
**int PORT,**
**----------**
**Gmail ==> Puerto 587 (25 o 465 pueden presentar problemas)**
**Yahoo ==> Puerto 587 (25 o 465 pueden presentar problemas)**
**Hotmail ==> Puerto 25 o 465**
**----------**

**Nota Importante:**
Es conveniente aclarar que los grandes proveedores de servicios de internet como GMAIL, HOTMAIL, etc., usualmente establecen determinado nivel de protección contra la emisión de mensajes por programa, por ejemplo en el caso de GMAIL, Google advierte siempre al destinatario que el mensaje recibido se considera implícitamente como SPAM de modo tal de forzar al emisor que oficialice tal servicio se desea enviar
mensajes programados a sus clientes o destinatarios.

string Login, (Login del usuario)
string Password, (Password del Login). En Gmail no sirve la contraseña normal de la cuenta: hay que activar la verificación en dos pasos y crear una "contraseña de aplicación" (ver support.google.com/accounts/answer/185833)
string SUBJECT, (Título del tema del mail)

string FROM, (Dirección mail del remitente)
string[] TO, (Dirección mail del destinatario)
string ATTACH, (Ficheros adjuntos a transmitir)
string[] MessageBody) (Cuerpo del mensaje)
RETORNO: NO TIENE

**public void SendMail(MailMessage MessageParameters)**

**Uso: Winforms App & WEB App**
Realiza el envío del mail basado en los parámetros definidos por el método MailSender

**PARAMETROS DE ENTRADA:**
**MailMessage** MessageParameters

**DESCRIPCION:**
El objeto **MessageParameters** contiene una estructura del tipo **MailMessage** contiene toda la información proporcionada previamente por el método **MailSender**.

**MÉTODOS DE TRABAJO CON FECHAS**

**Uso: Winforms App & WEB App**

**public int CheckDateUS(string Fecha)**

**DESCRIPCION:**

Este método realiza la verificación de una fecha en formato U.S ("mm/dd/aaaa").

**PARAMETROS DE ENTRADA:**

string Fecha (con formato "mm/dd/aaaa")

RETORNO tipo int:

1 - OK
2 - "dd" no es numérico
3 - "dd" fuera de rango

4 - "mm" no es numérico
5 - "mm" fuera de rango
6 - "aaaa" no es numérico
7 - Primer dígito de¨"aaaa" no es 1 ni 2
8 - Segundo dígito de¨"aaaa" no es 9 ni 0
9 - No tiene slash en posicion 3 o 6
10 - Longitud incorrecta
11- Mes de Febrero con más de 29 días en
    año bisiesto
12- Mes de Febrero con más de 28 días en
    año no bisiesto
13- Los meses de Abril, Junio, Septiembre,
    o Noviembre
    no deben tener más de 30 días

**public int CheckDateESP(string Fecha)**

**DESCRIPCION:**

Este método realiza la verificación de una fecha con formato ESN "dd/mm/aaaa"

**PARAMETROS DE ENTRADA:**

string Fecha ("dd/mm/aaaa")

**RETORNO:**

1 - OK
2 - "dd" no es numérico
3 - "dd" fuera de rango
4 - "mm" no es numérico
5 - "mm" fuera de rango
6 - "aaaa" no es numérico
7 - Primer dígito de¨"aaaa" no es 1 ni 2
8 - Segundo dígito de¨"aaaa" no es 9 ni 0
9 - No tiene slash en posicion 3 o 6
10 - Longitud incorrecta
11- Mes de Febrero con más de 29 días en
    año bisiesto
12- Mes de Febrero con más de 28 días en
    año no bisiesto
13- Los meses de Abril, Junio, Septiembre,

o Noviembre no deben tener más de 30 días

**public string  NormalizeDateUS(string FechaEntrada)**

**DESCRIPCION:**

Este método normaliza la fecha de entrada que aparece en la variable FechaEntrada en formato US (m/d/aaaa) no normalizado a formato US normalizado ("mm/dd/aaaa") retornando a este último como string.

Se considera normalizada la fecha cuando aparece expresado el año con cuatro dígitos, el mes con dos dígitos y el día con dos dígitos.

Ejemplo:

Si la entrada es: "1/2/1980", entonces retorna: "01/02/1980"

**PARAMETROS DE ENTRADA:**

string FechaEntrada

**RETORNO:**

string **(CON LA FECHA NORMALIZADA)**

Si la fecha de entrada es incorrecta se retorna la cadena "0".
Es conveniente aclarar que el año de entrada debe tener obligatoriamente 4 dígitos.

**public string NormalizeDateESP_Str(string FechaEntrada)**

**DESCRIPCION:**
Este método es similar al NormalizeDateUS pero asumiendo que la fecha de entrada tiene el

formato "dd/mm/aaaa"

**PARAMETROS DE ENTRADA:**
string FechaEntrada

**RETORNO:**
string **(CON LA FECHA NORMALIZADA)**

Si la fecha de entrada es incorrecta se retorna la cadena "0".
Es conveniente aclarar que el año de entrada debe tener obligatoriamente 4 dígitos.

**public string NormalizeDateISO_Str(string FechaEntrada)**

**DESCRIPCION:**
Este método normaliza la fecha de entrada que aparece en la variable **FechaEntrada** en formato ISO (aaaa-m-d) no normalizado a formato ISO normalizado (aaaa-mm-dd) retornando a este último.

Se considera normalizada la fecha cuando aparece expresado el año con cuatro dígitos, el mes con dos dígitos y el día con dos dígitos.

**PARAMETROS DE ENTRADA:**
string FechaEntrada

**RETORNO:**
string

Ejemplo:

Entrada: **1980-1-1**
Salida: **1980-01-01**

**public string ExtractHour_STR_HM(DateTime Fecha)**

DESCRIPCION:
Este método extrae la fecha del DateTime retornándola normalizada en formato de string.

PARAMETROS DE ENTRADA:
DateTime **Fecha**

RETORNO:

string ("0" si hubo error)

**public string NormalizeHour_STR(string HoraEntrada)**

Descripción:
Este método normaliza la hora de entrada que aparece en la variable **HoraEntrada** ("h:m:s") en formato normalizado ("hh:mm:ss").

PARAMETROS DE ENTRADA:
string **HoraEntrada**

Retorno:
string - (“0” si hubo error)
(“hh:mm:ss”) – Normalizada si todo OK

**public string ExtractDateUS_Str(DateTime Fecha)**

DESCRIPCION:
Este método extrae la fecha del parámetro de entrada con formato US (m/d/yyyy), retornándola normalizada (mm/dd/yyyy) en formato de string.

PARAMETROS DE ENTRADA:
DateTime **Fecha**

RETORNO:
string

**public string ExtractDateESP_Str(DateTime Fecha)**

DESCRIPCION:
Este método extrae la fecha del DateTime con formato ESP (dd/mm/yyyy) retornándola normalizada en formato de string.

PARAMETROS DE ENTRADA:
DateTime **Fecha**

RETORNO:
string

**public string ConvertDateUStoESP_Str(string FechaEntrada)**

**DESCRIPCION:**
Este método convierte la fecha de entrada que aparece en la variable FechaEntrada en formato US ("mm/dd/aaaa”) a formato ESP ("dd/mm/aaaa”) retornando a este último en formato de string.

Ejemplo:
Entrada: "1/5/1980" (5 de Enero de 1980 en formato US)
Salida:  "05/01/1980" (5 de Enero de 1980 en formato ESP)

**PARAMETROS DE ENTRADA:**
string FechaEntrada

**RETORNO:**
string (con la fecha en formato "dd/mm/aaaa")

Si la fecha de entrada es incorrecta se retorna la cadena "0".
Es conveniente aclarar que el año de entrada debe tener obligatoriamente 4 dígitos.

**public string ConvertDateESPtoUS_Str(string FechaEntrada)**

**DESCRIPCION:**
Este método convierte la fecha de entrada que aparece en la variable **FechaEntrada** en formato ESP ("dd/mm/aaaa”), a formato US ("mm/dd/aaaa”) retornando a este último en formato de string.

Ejemplo:
Entrada: "5/1/1980" (5 de Enero de 1980 en formato ESP)
Salida:  "01/05/1980" (5 de Enero de 1980 en formato US)

**PARAMETROS DE ENTRADA:**
string FechaEntrada

**RETORNO:**
string (con la fecha en formato "dd/mm/aaaa")

Si la fecha de entrada es incorrecta se retorna la cadena "0".
Es conveniente aclarar que el año de entrada debe tener obligatoriamente 4 dígitos.

**public string  ConvertDateISOtoUS_Str(string FechaEntrada)**

**DESCRIPCION:**
Este método convierte la fecha de entrada que aparece en la variable **FechaEntrada** en formato ISO (**"aaaa-mm-dd"**) a formato US (**"mm/dd/aaaa"**) retornando a este último con formato de **string**.

**PARAMETROS DE ENTRADA:**
string **FechaEntrada**

**RETORNO:**
string (con la fecha en formato **"mm/dd/aaaa"**)

**public string ConvertDateISOtoESP_Str(string FechaEntrada)**

DESCRIPCION:
Este método convierte la fecha de entrada que aparece en la variable FechaEntrada en formato ISO (aaaa-mm-dd) a formato español (dd/mm/aaaa), retornando a este último como string.

PARAMETROS DE ENTRADA:
string **FechaEntrada**

RETORNO:
string

**public int EdadIntESP(string FNac)**

**DESCRIPCION:**
Este método retorna la edad de una persona hasta la fecha como número entero, teniendo como argumento de entrada su Fecha de Nacimiento en formato de string y regionalización ESN (español="dd/mm/aaaa").

Tenga en cuenta que la fecha de entrada tiene que estar normalizada, es decir, dos dígitos para el día, dos dígitos para el mes y 4 dígitos para el año.

Ejemplo de uso:
LN.LN OBJ = new LN.LN(); // Instanciar al objeto OBJ de tipo LN
int ALFA = OBJ.EdadStrESP(@"28/06/1948");
quedando ALFA = 70 (Asumiendo que hoy es 22 de Septiembre del 2020)

**NOTA IMPORTANTE:**

Al inicio de la ejecución de este método se asegura que el formato de entrada sea español de México, el cual coincide con muchos formatos de fecha del tipo "dd/mm/aaaa" en Latinoamérica. Si se desea los valores posibles para el argumento de CultureInfo puede visitar en Internet el sitio:

https://msdn.microsoft.com/en-us/library/cc233982.aspx

Para Estados Unidos se usa "en-US" donde el formato de fecha es "mm/dd/aaaa".

**PARAMETROS DE ENTRADA:**
string **FNac**

**RETORNO:**
-1 ==> Si hubo error de formato en la fecha de entrada
edad ==> Si todo fué OK

**public double EdadDecESP(string FNac)**

**DESCRIPCION:**
Este método retorna la edad de una persona hasta la fecha como número decimal, teniendo como argumento de entrada su Fecha de Nacimiento en formato de string y regionalización española = "dd/mm/aaaa". A diferencia del método EdadESP, aquí se da la fecha exacta en forma decimal.

Tenga en cuenta que la fecha de entrada tiene que estar normalizada, es decir, dos dígitos para el día, dos dígitos para el mes y 4 dígitos para el año.

Ejemplo de uso:

```
using LN;
...
LN.LN OBJ = new LN.LN(); // Instanciar al objeto OBJ de tipo LN
double ALFA = OBJ.EdadDecESP(@"28/06/1948");
```

**NOTA IMPORTANTE:**
Al inicio de la ejecución de este método se asegura que el formato de entrada sea español de México, el cual coincide con muchos formatos de fecha del tipo "dd/mm/aaaa" en Latinoamérica. Si se desea los valores posibles para el argumento de CultureInfo puede visitar

en Internet el sitio:

https://msdn.microsoft.com/en-us/library/cc233982.aspx

Para Estados Unidos se usa "en-US" donde el formato de fecha es "mm/dd/aaaa".

**PARAMETROS DE ENTRADA:**
string **FNac**

**RETORNO:**
-1 ==> Si hubo error de formato en la fecha de entrada
edad ==> Si todo fue OK

**public int EdadIntUS(string FNac)**

**DESCRIPCION:**
Este método retorna la edad de una persona hasta la fecha como número entero, teniendo como argumento de entrada su Fecha de Nacimiento en formato de string y regionalización US ("**mm/dd/aaaa**"). Tenga en cuenta que la fecha de entrada tiene que estar normalizada, es decir, dos dígitos para el mes, dos dígitos para el día y 4 dígitos para el año.

Ejemplo de uso:

```
LN.LN OBJ = new LN.LN(); // Instanciar al objeto OBJ de tipo LN
int ALFA = OBJ.EdadIntUS(@"06/28/1948");
```

quedando ALFA = 70 (Asumiendo que hoy es 16 de diciembre del 2019)

**NOTA IMPORTANTE:**

Al inicio de la ejecución de este método se asegura que el formato de entrada sea inglés de Estados Unidos.

Si se desea los valores posibles para el argumento de CultureInfo puede visitar en Internet el sitio:

https://msdn.microsoft.com/en-us/library/cc233982.aspx

Para Estados Unidos se usa "en-US" donde el formato de fecha es "mm/dd/aaaa".

**PARAMETROS DE ENTRADA:**
string **FNac**

**RETORNO:**
-1 ==> Si hubo error de formato en la fecha de entrada
edad ==> Si todo fue OK

**public int EdadDecUS(string FNac)**

**DESCRIPCION:**
Este método retorna la edad de una persona hasta la fecha como número decimal, teniendo como argumento de entrada su Fecha de Nacimiento en formato de string y regionalización US ("mm/dd/aaaa"). Tenga en cuenta que la fecha de entrada tiene que estar normalizada, es decir, dos dígitos para el mes, dos dígitos para el día y 4 dígitos para el año.

Ejemplo de uso:
LN.LN OBJ = new LN.LN(); // Instanciar al objeto OBJ de tipo LN
int ALFA = OBJ.EdadDecUS(@"06/28/1948");

quedando ALFA en formato de número decimal.

**NOTA IMPORTANTE:**

Al inicio de la ejecución de este método se asegura que el formato de entrada sea inglés de USA. Si se desea los valores posibles para el argumento de CultureInfo puede visitar en Internet el sitio:

https://msdn.microsoft.com/en-us/library/cc233982.aspx

Para Estados Unidos se usa "en-US" donde el formato de fecha es "mm/dd/aaaa".

**PARAMETROS DE ENTRADA:**
string **FNac**

**RETORNO:**
-1 ==> Si hubo error de formato en la fecha de entrada
edad ==> Si todo fué OK

**public int EdadDesdeHastaESP(string Desde, string Hasta)**

**DESCRIPCION:**
Este método retorna la diferencia **Hasta - Desde** en años como entero, teniendo como argumentos de entrada la fecha **Desde** y **Hasta** en formato string.

Es requisito que la fecha (**Hasta** sea posterior a **Desde**, si esto no se cumple o si existe algún otro error en los parámetros de entrada entonces se retorna: -1

En este método se asume que las fechas de entrada están en regionalización española (dd/mm/aaaa).

**PARAMETROS DE ENTRADA:**
string **Desde**
string **Hasta**

**RETORNO:**
int

**Ejemplo de uso:**
using LN;
...
LN.LN OBJ = new LN.LN();
. . .
int ALFA = OBJ.EdadDesdeHastaESP(@"28/06/1948", @"16/12/2019");
da un resultado en ALFA como número entero.

**public int EdadDesdeHastaUS(string Desde, string Hasta)**

**DESCRIPCION:**
Este método Retorna la edad de una persona como entero teniendo como argumentos de entrada su Fecha de Nacimiento en formato string y la fecha de **Hasta** en formato string.

Es requisito que la fecha (Hasta sea posterior a **Desde**, si esto no se cumple o si existe algún otro error en los parámetros de entrada entonces se retorna: -1

En este método se asume que las fechas de entrada están en regionalización USA (mm/dd/aaaa).

**PARAMETROS DE ENTRADA:**
string **Desde**

string **Hasta**

**RETORNO:**
int

**Ejemplo de uso:**

```
using LN;
...
LN.LN OBJ = new LN.LN();
. . .
int ALFA = OBJ.EdadDesdeHastaUS(@"06/28/1948", @"12/16/2019");
```

da como resultado:
ALFA = 70

**public double EdadDesdeHastaDecESP(string Desde, string Hasta)**

**DESCRIPCION:**
Este método Retorna la edad de una persona como decimal teniendo como argumentos de entrada su Fecha de Nacimiento en formato string y la fecha de **Hasta** en formato string.

Es requisito que la fecha (Hasta sea posterior a **Desde**, si esto no se cumple o si existe algún otro error en los parámetros de entrada entonces se retorna: -1

En este método se asume que las fechas de entrada están en regionalización España (¨dd/mm/aaaa¨).

**PARAMETROS DE ENTRADA:**
string **Desde**
string **Hasta**

**RETORNO:**
double

**Ejemplo de uso:**

```
using LN;
...
LN.LN OBJ = new LN.LN();
. . .
int ALFA = OBJ.EdadDesdeHastaUS(@"28/06/1948", @"16/12/2019");
```
da como resultado:
ALFA = 70.63

**public double EdadDesdeHastaDecUS(string Desde, string Hasta)**

**DESCRIPCION:**
Este método Retorna la edad de una persona como decimal teniendo como argumentos de entrada **Desde** en formato string y **Hasta** en formato string.

Es requisito que la fecha **Hasta** sea posterior a **Desde**, si esto no se cumple o si existe algún otro error en los parámetros de entrada entonces se retorna: -1

En este método se asume que las fechas de entrada están en regionalización USA (¨mm/dd/aaaa¨).

**PARAMETROS DE ENTRADA:**
string **Desde**
string **Hasta**

**RETORNO:**
int

**Ejemplo de uso:**

```
using LN;
...
LN.LN OBJ = new LN.LN();
. . .
int ALFA = OBJ.EdadDesdeHastaUS(@"06/28/1948", @"12/16/2019");
```

da como resultado:
ALFA = 70.63

**public double DateDifD(DateTime DESDE, DateTime HASTA)**

**DESCRIPCION:**

Método que haya la diferencia en días entre las fechas DESDE y HASTA
(HASTA - DESDE).

**Parámetros de Entrada:**
DateTime **DESDE**
DateTime **HASTA**

**Retorno:**
-1 **HASTA** no es posterior a **DESDE**.
-1 Los parámetros de entrada no pueden ser nulos.

Un número decimal mayor o igual que 0 si todo fue correcto, donde este número representa la diferencia en días de HASTA - DESDE.

**NOTA:**
Cuando se quiere introducir una fecha específica para DESDE o HASTA, se puede hacer como mostramos a continuación:

DateTime DESDE = new DateTime(2018, 08, 25);

Y entonces la fecha quedaría en DESDE bajo formato DateTime de C#.

**public double DateDifH(DateTime DESDE, DateTime HASTA)**

**DESCRIPCION:**
Método que haya la diferencia en horas entre las fechas **DESDE** y **HASTA**
(**HASTA** - **DESDE**).

**Parámetros de Entrada:**
DateTime **DESDE**
DateTime **HASTA**

**Retorno:**
-1 **HASTA** no es posterior a **DESDE**.
-1 Los parámetros de entrada no pueden ser nulos.

Un número decimal mayor o igual que 0 si todo fue correcto, donde este número representa

la diferencia en horas de **HASTA** - **DESDE**.

**NOTA:**
Cuando se quiere introducir una fecha específica para DESDE o HASTA, se puede hacer como mostramos a continuación:

DateTime **DESDE** = new DateTime(2018, 08, 25);

Y entonces la fecha quedaría en **DESDE** bajo formato DateTime de C#.

**public double DateDifM(DateTime DESDE, DateTime HASTA)**

**DESCRIPCION:**

Método que haya la diferencia en meses entre las fechas **DESDE** y **HASTA**
(**HASTA** - **DESDE**).

**Aclaración Importante:**
Dado que el número de días de los meses puede variar en el caso de Febrero en dependencia de que el año sea bisiesto o no, hacemos una consideración especial de asumir todos los meses de 30 días.

**Parámetros de Entrada:**
DateTime **DESDE**
DateTime **HASTA**

**Retorno:**
-1 **HASTA** no es posterior a **DESDE**.
-1 Los parámetros de entrada no pueden ser nulos.

Un número decimal mayor o igual que 0 si todo fue correcto, donde este número representa la diferencia en meses de **HASTA** - **DESDE**.

**NOTA:**
Cuando se quiere introducir una fecha específica para **DESDE** o **HASTA**, se puede hacer como mostramos a continuación:

DateTime **DESDE** = new DateTime(2018, 08, 25);

Y entonces la fecha quedaría en **DESDE** bajo formato DateTime de C#.

**public bool NormalizeDate(int A, int M, int D, ref int ANorm, ref int MNorm, ref int DNorm)**

**Descripción:**
Esta función realiza la normalización de las cantidades dadas en los primeros tres parámetros de modo tal que si la cantidad de meses de un año es mayor que 12 o si la cantidad de días es mayor que 30 se realiza el ajuste necesario quedando el resultado en las variables de salida ANorm, MNorm, y DNorm de tipo **integer**. La transmisión a estos últimos se hace por referencia y de esta forma se retorna más de un valor.

Parámetros de entrada:
int A
int M
int D
ref int ANorm
ref int MNorm
ref int DNorm

Retorno:
ref int ANorm ==> Número de años ya normalizado
ref int MNorm ==> Número de meses ya normalizado
ref int DNorm ==> Número de días ya normalizado

Ejemplo:

```
using LN;
...
LN.LN OBJ = new LN.LN();
int AA = 0;
int MM = 0;
int DD = 0;
bool Result;
...
Result = OBJ.NormalizeDatesESP(0,135,105,ref AA, ref BB, ref DD);
...
```

Retorno:
En este ejemplo se considera que hay 0 años, 135 meses y 105 días que dan como resultado normalizado, 11 años, con 6 meses y 15 días.

true  - Si todo fué OK

En este caso quedaría:
AA = 11
BB = 6
CC = 15

false - Si hubo error

**MÉTODOS DE VALIDACIÓN DE DATOS**
**Uso: Winforms App & WEB App**

**public bool SSNCheck(string SSN)**

**DESCRIPCION:**
Esta función valida la cadena de entrada SSN que representa el Social Security Number de una persona en USA. El chequeo comprende solamente la sintaxis del formato general, pero no la semántica. Es decir, solo se chequea que inicialmente aparezcan tres dígitos, seguidos de un guion, y finalmente cuatro dígitos más.

Formato General:
**999-99-9999**

**PARAMETROS DE ENTRADA:**

string  SSN (Social Security)

**RETORNO:**
**true** – Si cumple la condición
**false** – Si no cumple la condición

**public bool NUM(string Numero)**

**DESCRIPCION:**

Este método chequea si un string es un número entero sin signo, retornando <**true**> en caso afirmativo y <**false**> si no lo es.

**PARAMETROS DE ENTRADA:**

string **Numero**

**RETORNO:**
**true** – Si cumple la condición
**false** – Si no cumple la condición

**public bool NUMERIC(string Numero)**

**DESCRIPCION:**
Este método chequea si el parámetro de entrada es numérico en cualquiera de sus variantes de representación de enteros o decimales (+11.23, -1.5, 4.2, -.123, +.6932, +321, -4, 56., -43., 2391, 001, etc.).

**PARAMETROS DE ENTRADA:**
string **Numero**

**RETORNO:**
**true** – Si cumple la condición
**false** – Si no cumple la condición

**public bool ZERO(string Numero)**

**DESCRIPCION:**
Este método chequea si el parámetro de entrada contiene ceros solamente retornando <**true**> en caso afirmativo.

**PARAMETROS DE ENTRADA:**
string **Numero**

**RETORNO:**
**true** - Si **Numero** contiene solamente ceros.
**false** - Caso contrario

**public bool ALFANUM(string Cadena)**

**DESCRIPCION:**
Este método chequea si un string es alfanumérico retornando <**true**> en caso afirmativo y <**false**> si no lo es. Este método considera que una cadena es alfanumérica cuando contiene solo letras y números.
**PARAMETROS DE ENTRADA:**
string **Cadena**
**RETORNO:**
**true** - Si la cadena es alfanumérica
**false** - Si no lo es

**public bool ALF(string Cadena)**

**DESCRIPCION:**
Este método chequea si un string es alfabético bajo el alfabeto español donde existen las letras acentuadas, la ñ, etc., retornando <**true**> en caso afirmativo y <**false**> si no lo es.

Se considera que una cadena es alfabética en nuestro alfabeto cuando contiene solo letras o aparezcan los caracteres '-', ó apostrofo (') intermedios y que aparezcan una sola vez.

**PARAMETROS DE ENTRADA:**
string **Cadena**

**RETORNO:**
**true** - Si la cadena es alfabética
**false** - Si no lo es

**Ejemplo:**
Domínguez, Ramírez, Roberto O'Farril Sinforoso

**public bool ALFEng(string Cadena)**

**DESCRIPCION:**
Este método chequea si un string es alfabético bajo el alfabeto inglés, pero con la característica de que no deben aparecer acentos, la letra ñ, ni las diéresis del alfabeto español.

**PARAMETROS DE ENTRADA:**
string **Cadena**

**RETORNO:**
**true** - Si la cadena es alfabética
**false** - Si no lo es

Ejemplo:
Casos correctos:
DOMINGUEZ, O'DONNEL, RAMIREZ, ROBERTO, O'FARRIL SINFOROSO

Casos Incorrectos:
DOMÍNGUEZ, RAMÍREZ, ZUÑIGA

**public bool Bisiesto(string AAAAStr)**

**DESCRIPCION:**
Función que determina si el año de entrada es bisiesto, recordar que en los años bisiestos febrero tiene 29 días y en el resto solo 28.

**PARAMETROS DE ENTRADA:**
string **AAAAStr**

**RETORNO:**
**true** – Si cumple la condición
**false** – Si no cumple la condición

**public bool BetweenInc(int N, int a, int b)**

**Descripción:**
Método que determina si el número entero N se encuentra en el intervalo cerrado [a,b] (incluye los extremos).

**Parámetros de entrada:**
int **N**
int **a**
int **b**

**Retorno:**
**true** - Si se encuentra en el intervalo
**false** - Si no se encuentra en el intervalo

**public bool BetweenExc(int N, int a, int b)**

**Descripción:**
Método que determina si el número entero N se encuentra en el intervalo abierto (a,b) (excluye los extremos).

**Parámetros de entrada:**
int **N**
int **a**
int **b**

**Retorno:**
**true** - Si se encuentra en el intervalo
**false** - Si no se encuentra en el intervalo

**public bool BetweenIncDec(double N, double a, double b)**

**Descripción:**
Método que determina si el número decimal N se encuentra en el intervalo cerrado [a,b] (incluye los extremos).

**Parámetros de entrada:**
int **N**
int **a**
int **b**

**Retorno:**
**true** - Si se encuentra en el intervalo
**false** - Si no se encuentra en el intervalo

**public bool BetweenExcDec(double N, double a, double b)**

**Descripción:**
Método que determina si el número decimal N se encuentra en el intervalo abierto (a,b) (excluye los extremos).

**Parámetros de entrada:**
int N
int a
int b

**Retorno:**
**true** - Si se encuentra en el intervalo
**false** - Si no se encuentra en el intervalo

**bool VINCheck(string VIN)**

**Descripción:**
Esta función valida la cadena de entrada VIN de un carro que representa su ID del fabricante con 17 car.
**Nota:**
A partir del año 1980 se adoptó internacionalmente asignar un identificador único a cada auto, el cual tiene una ley de formación que contiene el ID del constructor, y muchos otros parámetros. Si Ud. Desea saber más acerca de este identificador puede consultar en internet.

**Parámetros de entrada:**
string **VIN** (ID del Carro)

**RETORNO:**
**true** – Si el **VIN** está correcto
**false** – Si es erróneo

**MÉTODOS DE EJECUCIÓN DE PROCESOS**
**Uso: Winforms App & WEB App**

**public int MSDOSWait(string comando)**

**Descripción:**
Este método ejecuta un comando MS-DOS en una ventana de consola quedando en espera de la ejecución del comando para continuar sin cerrar dicha ventana en espera de que termine la ejecución del comando invocado.

**Parámetros de entrada:**
string **comando**

**RETORNO:**
Código de terminación del proceso invocado (**ExitCode**)

**public int MSDOSNoWait(string comando)**

**Descripción:**
Este método ejecuta un comando MS-DOS en una ventana de consola, pero no queda en espera como el caso anterior. Por esta razón la tarea se ejecuta asincrónicamente respecto al programa llamador.

**PARAMETROS DE ENTRADA:**
string **comando**

**RETORNO:**
Código de terminación del proceso invocado (**ExitCode**)

**public void PAUSE()**

**DESCRIPCION:**
Provoca una pausa en la consola de comandos, (PAUSE) del MS-DOS emitiendo el siguiente mensaje:

***** PRESIONE CUALQUIER TECLA PARA CONTINUAR *****

**PARAMETROS DE ENTRADA:**
No tiene

**RETORNO:**
No tiene

**public int ExecProcess(string comando)**

**DESCRIPCION:**

Este método permite invocar a cualquier objeto de Windows (cualquier extensión reconocida por el sistema), por tanto, si invocamos a un fichero .docx y tenemos al Office instalado, pues el Windows abre al fichero con Word, y así sucederá con cualquier otra extensión reconocida por el sistema.  Con este método es también posible llamar a cualquier tipo de ejecutable (.exe, .bat, .msi, msc, etc.).

La ejecución con este método es secuencial, de modo que hasta que no finalice el proceso con el objeto invocado no se pasa a la siguiente instrucción de C#.

A continuación, mostramos tres ejemplos de llamadas.

**Ejemplo1:**

Abrir al documento "ReporteMensual.docx" que se encuentra dentro de la carpeta INFO de D:\

```
OBJLN.ExecProcess(@"D:\INFO\ReporteMensual.docx");
```

**Ejemplo2:**

(Ejecución de un .bat que se encuentra en la raíz de D:\

```
OBJLN.ExecProcess(@"D:\ABC.bat");
```

**Ejemplo3:**

Ejecución de un .exe que aparece dentro de una carpeta cuyo nombre contiene espacios, en este caso el PATH debe estar entre comillas y siguiendo la sintaxis de C# por cada comilla que se desee insertar en el string de especificación, hay que poner dos, por tal razón aparecen tres comillas al inicio y al final.

Si Ud. no hace esto el ejecutable referenciado no ejecutará
correctamente.

```
OBJLN.ExecProcess(@"""C:\Renamer 2.20\ReNamer.exe""");
```

PARAMETROS DE ENTRADA:
string comando

RETORNO:
Código de terminación.

**public int ExecProcessNoWait(string comando)**

**DESCRIPCION:**
Este método es similar al ExecProcess pero no es secuencial como el anterior que espera a la terminación del proceso para continuar, aquí se deja al proceso invocado en ejecución y se continua con las subsiguientes instrucciones C# del programa.

**PARAMETROS DE ENTRADA:**
string **comando**

**RETORNO:**
Código de terminación.

**MÉTODOS DE FUNCIONES DE CADENA DE CARACTERES**
**Uso: Winforms App & WEB App**

Antes de comenzar con la descripción de cada método de esta sección es conveniente aclarar que la mayoría están programados bajo el uso de Expresiones Regulares. Si el lector no está familiarizado con las mismas lo remitimos a la documentación del utilitario Regex Test desarrollado por el autor y que también está a disposición del que lo desee en Windows Store.

**public int FindStr(string Cadena, string Subcadena)**

**DESCRIPCION:**
Este método calcula las veces que <**Subcadena**> está contenida dentro de <**Cadena**>.

Ejemplos:
Nveces = OBJ.FindStr(**Cadena**, ":")

En este caso, si cadena = "**05:03:20**" entonces Nveces = 2, dado que el caracter ":" está contenido dos veces dentro de **Cadena**.

**PARAMETROS DE ENTRADA:**
string **Cadena**
string **Subcadena**

**RETORNO:**
Int

**public string FillString(string Car, int Nveces)**

**DESCRIPCIÓN:**
Propaga al carácter <**Car**> (entre comillas como cadena) **Nveces** retornando un string con el resultado de la propagación.

**PARAMETROS DE ENTRADA:**
string **Car**
int **NVeces**

**Retorno:**
Una cadena conteniendo a Car N veces.

**Ejemplo:**

```
string CAD = "";
string CAR = "*";
int N = 5;
LN.LN OBJ = new LN.LN();
...
CAD = OBJ.FillString(CAR, N);
```

Esto deja a CAD = "*****";

**public string DeleteChar(string Cadena, string CHAR)**

**DESCRIPCION:**
Esta función elimina las apariciones de los caracteres de la subcadena **CHAR** dentro de **Cadena**.

**PARAMETROS DE ENTRADA:**
string Cadena

string CHAR
**RETORNO**: string

**Ejemplos:**
**Ejemplo 1:**
Si Phone = "(786)905-1265" e invocamos a la función de la forma siguiente:
string Resultado = DeleteSTR(Phone, "()-");

esto significa que queremos borrar las apariciones de los caracteres "()-" que estén dentro de Phone y el resultado quedaría así:
Resultado = "7869051265"

Nota: Es conveniente aclarar que en este método se realiza la búsqueda de cada uno de los caracteres "()-" independiente del orden en que aparezcan, por tal razón en el ejemplo anterior se podría haber especificado como patrón esto: "-)(", y el resultado sería el mismo dado que lo que estamos diciendo es que dondequiera que se encuentre algunos de dichos caracteres, bórrelos.

**Ejemplo 2:**

Si CAD = "123:456:789" e invocamos a la función de la forma siguiente:

string Resultado = DeleteChar(CAD, ":");

esto significa que queremos borrar las apariciones del carácter ":" que esté dentro de CAD y el resultado quedaría así:

Resultado = "123456789"

**public string DeleteStr(string Cadena, string PATRON)**

**DESCRIPCION**:
Esta función elimina las apariciones de la subcadena STR dentro de Cadena retornando la cadena resultante.

**PARAMETROS DE ENTRADA:**
string **Cadena**
string **PATRON**
RETORNO:
string

Ejemplo 1:
Si CAD = "123abc4567abc890" e invocamos a la función de la forma siguiente:
string Resultado = DeleteSTR(CAD, "abc");
esto significa que queremos borrar las subcadenas "abc" que estén dentro de CAD y el resultado quedaría así:

Resultado = "1234567890"

**public void SplitString(ref string[ ]** Resultado**, string** Cadena**, string[ ]** delimitador**)**

**DESCRIPCION:**
Extrae las subcadenas separadas por los delimitadores especificados en el tercer parámetro del string que se encuentra contenido en **Cadena**, dejando las subcadenas en el arreglo de tipo string **Resutado**, donde la transmisión es por referencia tal y como se muestra a continuación.

**Ejemplos de Uso:**

**Caso1:**

En la secuencia que aparece a continuación, se toma como único delimitador la coma.

string[] RESULT = {""};

* La parte derecha inicializa el arreglo RESULT, y esto es obligatorio antes de usar la función SplitString

string[] comas = new string[] {","}; -> Define el separador como coma

string source = "UNO,DOS,TRES,CUATRO";

OBJ.SplitString(ref RESULT, source, comas);

En este caso quedaría en el array RESULT lo siguiente:
RESULT[0] = "UNO";
RESULT[1} = "DOS";
RESULT[2} = "TRES";
RESULT[3} = "CUATRO";

**Caso 2:**

En la secuencia que aparece a continuación, se toman varios delimitadores, la coma, un punto y un punto y coma.

```
string[] delim = new string[] {",",".",";"};
string source = "UNO,DOS.TRES;CUATRO";

OBJ.SplitString(ref RESULT, source, delim);
```

En este caso quedaría en el array RESULT lo siguiente:

```
RESULT[0] = "UNO";
RESULT[1] = "DOS";
RESULT[2] = "TRES";
RESULT[3] = "CUATRO";
```

Parámetros de Entrada:

```
ref string[] Resultado,
ref delim    Cadena,
string[]     delimitador
```

Retorno:
No Tiene

**METODOS GENERALES**

**Uso: Winforms App & WEB App**

**public double Porciento(double P, double N)**

**DESCRIPCION:**
Este método retorna el porciento **P** del número **N**. Por ejemplo, si queremos saber el 50% de 20 invocaríamos a la función de la forma siguiente:

```
double result = OBJ.Porciento(50,20)

entonces result = 10
```

**PARAMETROS DE ENTRADA:**
**double** P
**double** N

**RETORNO:**
double

**public string Trunc(double Origen, int Enteros, int Decimales)**

**Descripción:**
Esta función toma el contenido double de la variable Origen y lo representa con la cantidad de cifras de la variable Enteros para la parte entera y la cantidad de cifras de Decimales para la parte decimal.

**Retorno:**
Un string que contiene el valor double de origen y lo representa en el número de dígitos que contiene Enteros y el número de dígitos que contiene Decimales. La parte entera se rellena con ceros a la izquierda si es necesario.

**Ejemplo:**

Si **Origen** = 12.33444
**Enteros** = 5
**Decimales** = 2

Entonces:

```
LN.LN OBJ = new LN.LN();
...
OBJ.Trunc(12.33444, 5, 2)
```

retorna el siguiente string:
"00012.33"

**public void Upper(ref string Cadena)**

**DESCRIPCION:**
Convierte a mayúsculas la cadena del parámetro de entrada. La transmisión de parámetros es por referencia.
Ejemplo:

LN.LN OBJ = new LN.LN();
string CAD = "abc";
...
**OBJ.Upper(ref CAD);**
Después de la ejecución del método **Upper** quedaría:
CAD = "ABC"

**Parámetros de Entrada:**
string **Cadena**
**Retorno:**
No Tiene

**public void Lower(ref string Cadena)**

**DESCRIPCION:**
Convierte a minúsculas la cadena del parámetro de entrada. La transmisión de parámetros es por referencia.

**Ejemplo:**
LN.LN OBJ = new LN.LN();
string CAD = "ABC";
...

OBJ.Lower(**ref** CAD);  // **ref** indica que la transmisión es por referencia

Después de la ejecución del método Lower CAD quedaría así:
CAD = "abc"

**Parámetros de Entrada:**
**Cadena**

Retorno:
**No Tiene**

**public void ShowRPT(string PathReport)**
Programadores: Geovany González y Héctor Fortún

**Descripción:**
El método **ShowRPT** está diseñado con el objetivo de visualizar rápidamente un reporte estático (.RPT) emitido por el utilitario **Crystal Report** autónomo. Es necesario aclarar que los reportes generados dinámicamente por programa con el uso de los métodos del **Crystal**

**Report** como **SetParameterValue**am, **ExportToDisk, etc.**, no pueden ser emitidos mediante **ShowRPT.**

Este método se encarga de abrir una ventana autónoma y mostrar el contenido del reporte bajo el uso del **Viewer** del **Crystal Report** ahorrándole al programador una gran cantidad de tiempo de programación y de código ya que **ShowRPT** se encarga de todo.

EJEMPLO:
Suponiendo que previamente hemos generado el .RPT con la aplicación autónoma del Crystal Report en el path que sigue:

"D:\REPORTES**ReporteMensual.rpt**

Pues si queremos que nuestro programa C# muestre el contenido
de dicho reporte con el Crystal Viewer, bastaría con poner el siguiente código:

```
LN.LN OBJ = new LN.LN(); // Instanciar un objeto de LN
. . .
OBJ.ShowRPT(@"D:\REPORTES\ ReporteMensual.rpt ");
```

*** ASI DE FÁCIL ***

Parámetros de Entrada:
string **PathReport**

Retorno: No Tiene

**METODOS DE TRABAJO CON DIRECTORIOS**

**Método FileDiskOpenCreate**

**FORMATO:**

public **StreamWriter** FileDiskOpenCreate(string Path)

**DESCRIPCION:**

Este método recibe como entrada una cadena de caracteres que describe el directorio donde vamos a crear el fichero descrito en **Path**.

Si dicho directorio contiene sub-directorios, estos no se obtienen por este método ya que para este fin se dispone del método:

**FoldersDiskOpenCreate**
que se describe más adelante.

**PARAMETROS DE ENTRADA:**

string Path (contiene el full path del fichero a crear).

Nota: Recomendamos que use @"path_del_fichero" a fin de usar los backslash (\) normales, sin necesidad de ponerlos duplicados en cada aparición.

**Ejemplo:**

```
. . .
StreamWriter FileOut;
. . .

FileOut = FileDiskOpenCreate(@"D:\Empleados\Departamento1\Dep1.txt");
```

**RETORNO:**

1) Retorna **null** si se presenta algún error en la apertura del fichero.

2) Retorna un elemento del tipo StreamWriter que debe ser referenciado por el método **FileDiskWrite**
que es el encargado de escribir un artículo en el fichero seleccionado y que se describe más adelante.

**Método FileDiskOpenAppend**

**FORMATO:**

public **StreamWriter** FileDiskOpenAppend(string Path)

**DESCRIPCION:**

Este método recibe como entrada una cadena de caracteres que describe el directorio que contiene el fichero donde vamos a adicionar los nuevos artículos mediante el método **FileDiskWrite** que se describe más adelante.

Si dicho directorio contiene sub-directorios, estos no se obtienen por este método ya que para este fin se dispone del método:

**FoldersDiskOpenCreate**
que se describe más adelante.

**PARAMETROS DE ENTRADA:**

string Path (contiene el full path del fichero al que se van a adicionar nuevos artículos).

Nota: Recomendamos que use @"path_del_fichero" a fin de usar los backslash (\) normales, sin necesidad de ponerlos duplicados en cada aparición.

**Ejemplo:**

```
. . .
StreamWriter FileOut;
. . .
FileOut = FileDiskOpenCreate(@"D:\Empleados\Departamento1\Dep1.txt");
```

**RETORNO:**

1) Retorna **null** si se presenta algún error en la apertura del fichero.

2) Retorna un elemento del tipo StreamWriter que debe ser referenciado por el método **FileDiskWrite**
que es el encargado de escribir/adicionar un nuevo artículo en el fichero seleccionado y que se describe más adelante.

**Método FileDiskOpenRead**

**FORMATO:**

public StreamReader FileDiskOpenRead(string Path)

**DESCRIPCION:**

Este método recibe como entrada una cadena de caracteres que describe el directorio de donde vamos a leer el fichero descrito en **Path**.

**PARAMETROS DE ENTRADA:**

string Path (contiene el full path del fichero desde donde se va a leer).

Nota: Recomendamos que use @"path_del_fichero" a fin de usar los backslash (\) normales, sin necesidad de ponerlos duplicados en cada aparición.

**Ejemplo:**

. . .
**StreamReader** FileIn;
. . .

FileIn = FileDiskOpenRead(@"D:\Empleados\Departamento1\Dep1.txt");

**RETORNO:**

1) Retorna **null** si se presenta algún error en la apertura del fichero.

2) Retorna un elemento del tipo StreamReader que debe ser referenciado por el método **FileDiskRead** que es el encargado de leer un artículo en el fichero seleccionado y que se describe más adelante.

**Método FileDiskWrite**

**FORMATO:**

public bool FileDiskWrite(StreamWriter OutFile, string Line)

**DESCRIPCION:**

Este método realiza la escritura del artículo contenido en **Line** en dependencia de como fué abierto dicho fichero (Create o Append).

**PARAMETROS DE ENTRADA:**

1) StreamWriter OutFile:

Aquí en OutFile se consigna la variable del tipo de datos StreamWriter que se obtuvo en el retorno de alguno de los métodos **FileDiskOpenCreate o FileDiskOpenAppend.**

2) string Line:

La variable Line contiene el artículo en formato de string que se desea escribir en el fichero abierto

**Ejemplo:**

```
. . .
string Artículo = "";
StreamWriter FileOut;
. . .
FileOut = FileDiskOpenCreate(@"D:\Empleados\Departamento1\Dep1.txt");
if       (FileOut == null)
          {
          *** Secuencia de tratamiento de error
          }
. . .
Artículo = "Expediente=015,Nombre=Leslie Sullivan";
FileDiskWrite(FileOut, Artículo);  // Escribe el artículo anterior en el fichero Dep1.txt
. . .
. . .
```

**RETORNO:**

1) Retorna **false** si se presenta algún error en la escritura del artículo.

2) Retorna **true** si todo fué OK.

**Método FileDiskRead**

**FORMATO:**

public string FileDiskRead(StreamReader InFile)

**DESCRIPCION:**

Este método realiza la lectura del artículo en curso retornándolo en formato string.
Este método esta concebido para ficheros .TXT (no aplica para ficheros binarios).

**PARAMETROS DE ENTRADA:**

1) StreamReader InFile:

Aquí en InFile se consigna la variable del tipo de datos StreamReader que se obtuvo en el retorno del método **FileDiskOpenRead.**

**Ejemplo:**

```
. . .
string Artículo = "";
StreamReader FileIn;
. . .
FileIn = FileDiskOpenRead(@"D:\Empleados\Departamento1\Dep1.txt");
if       (FileIn == null)
          {
          *** Secuencia de tratamiento de error
          }
. . .
// Recibe el artículo en curso dentro del fichero Dep1.txt depositándolo en Artículo
Artículo = FileDiskRead(FileIn);
. . .
. . .
```

**RETORNO:**

1) Retorna **null** si se presenta algún error en la escritura del artículo.

2) Retorna el artículo en curso si todo fué OK.

**Método FileDiskCloseWriter**

**FORMATO:**

public bool FileDiskCloseWriter(StreamWriter OutFile)

**DESCRIPCION:**

Este método realiza el cierre del fichero definido en **FileDiskOpenCreate** o **FileDiskOpenAppend**

**PARAMETROS DE ENTRADA:**

1) StreamWriter OutFile:

Aquí en OutFile se consigna la variable del tipo de datos StreamWriter que se obtuvo en el retorno de alguno de los métodos **FileDiskOpenCreate o FileDiskOpenAppend.**

**Ejemplo:**

```
. . .
string Artículo = "";
StreamWriter FileOut;
. . .
FileOut = FileDiskOpenCreate(@"D:\Empleados\Departamento1\Dep1.txt");
if       (FileOut == null)
          {
          *** Secuencia de tratamiento de error
          }
. . .
Artículo = "Expediente=015,Nombre=Leslie Sullivan";
FileDiskWrite(FileOut, Artículo);  // Escribe el artículo anterior en el fichero Dep1.txt
. . .
. . .
FileDiskCloseWriter(FileOut);
. . .
```

**RETORNO:**

1) Retorna **false** si se presenta algún error en la escritura del artículo.

2) Retorna **true** si todo fué OK.

**Método FileDiskCloseReader**

**FORMATO:**

public bool FileDiskCloseReader(StreamReader IntFile)

**DESCRIPCION:**

Este método realiza el cierre del fichero definido en **FileDiskOpenRead.**

**PARAMETROS DE ENTRADA:**

1) StreamReader InFile:

Aquí en InFile se consigna la variable del tipo de datos StreamReader que se obtuvo en el retorno del método **FileDiskOpenRead.**

**Ejemplo:**

```
. . .
string Artículo = "";
StreamReader FileIn;
. . .
FileIn = FileDiskOpenReader(@"D:\Empleados\Departamento1\Dep1.txt");
if       (FileIn == null)
          {
          *** Secuencia de tratamiento de error
          }
. . .
 // Obtiene el artículo en curso del fichero Dep1.txt y lo deposita en Artículo
Artículo = FileDiskRead(FileIn);
. . .
. . .
FileDiskCloseReader(FileOut);
. . .
```

**RETORNO:**

1) Retorna **false** si se presenta algún error en la escritura del artículo.

2) Retorna **true** si todo fué OK.

**Método GetDirectoryFilesDialog**

**FORMATO:**

public string[] GetDirectoryFilesDialog(ref int NumeroDeFicheros)

**DESCRIPCION:**

Este método retorna la lista de **ficheros** seleccionados dentro del diálogo en un arreglo de tipo **string**[] dejando también el número de los mismos en la variable **NumeroDeFicheros**

(transmisión por referencia por lo que ref es necesario).

**PARAMETROS DE ENTRADA:**

ref int NumeroDeFicheros

Variable transmitida por referencia que obtiene el número de ficheros contenidos dentro del directorio seleccionado en el diálogo. Si el directorio seleccionado contiene subcarpetas, estas no se cuentan por este método pues para este fin se tiene a los métodos: **GetDirectoryFoldersDialog y GetDirectoryFoldersPath.**

**Ejemplo:**

```
. . .
int NumeroDeFicheros = 0;
. . .
string[] ListaDeFicheros = GetDirectoryFilesDialog(ref NumeroDeFicheros);
. . .
```

*Nota:
En este ejemplo la lista de ficheros obtenida se copia en el arreglo ListaDeFicheros[] definido por el programador, quedando la cantidad de los mismos en la variable entera del segundo parámetro **NumeroDeFicheros**.

**RETORNO:**

1) Retorna **null** si se presenta algún error en la apertura del fichero.

2) Retorna la lista de ficheros que pertenecen al directorio seleccionado en ListaDeFicheros[]
y en el segundo parámetro (**NumeroDeFicheros**) la cantidad de ficheros**.**

**Método GetDirectoryFilesPath**

**FORMATO:**

public string[] GetDirectoryFilesPath(string Path, ref int NumeroDeFicheros)

**DESCRIPCION:**

Este método retorna la lista de **ficheros** contenidos en el directorio descrito en **Path** en un arreglo de tipo **string**[] y el número de los mismos en la variable del segundo parámetro **NumeroDeFicheros** (transmisión por referencia por lo que ref es necesario).

**PARAMETROS DE ENTRADA:**

string Path
Es un string que define el Path del directorio del cual queremos obtener su lista de ficheros

ref int NumeroDeFicheros
Variable transmitida por referencia que obtiene el número de ficheros del directorio escogido.

**Ejemplo:**

```
. . .
int NumeroDeFicheros = 0;
. . .
string[] ListaDeFicheros = GetDirectoryFilesPath(@"D:\Empleados\Dep\Dep1.txt",
                                                 ref NumeroDeFicheros);
. . .
```

*Nota:
En este ejemplo la lista de ficheros obtenida se copia en el arreglo ListaDeFicheros[] definido por el programador, quedando la cantidad de los mismos en la variable entera del segundo parámetro **NumeroDeFicheros**.

**RETORNO:**

1) Retorna **null** si se presenta algún error en la apertura del fichero.

2) Retorna  la lista de ficheros que pertenecen al directorio seleccionado y en el segundo parámetro (**NumeroDeFicheros**) la cantidad de ficheros de dicha lista**.**

**Método GetDirectoryFoldersDialog**

**FORMATO:**

public string[] GetDirectoryFoldersDialog(ref int NumeroDeSubDirectorios)

**DESCRIPCION:**
Este método obtiene la lista de **subfolders** contenidos dentro del directorio escogido por el usuario**,** retornando el resultado en un arreglo de tipo **string**[] y depositándo también el número de subfolders en la variable del parámetro **NumeroDeSubDirectorios** (transmisión por referencia por lo que ref es necesario).

**PARAMETROS DE ENTRADA:**

ref int NumeroDeSubDirectorios

Aqui queda después de la ejecución del método el número de subdirectorios contenidos en la carpeta seleccionada.

**Ejemplo:**

```
. . .
int NumeroDeFolders = 0;
string[] ListaSubFolders = null;
. . .
ListaSubFolders = GetDirectoryFoldersDialog(ref NumeroDeFolders);
. . .
```

*Nota:
En este ejemplo la lista de subfolders obtenida se copia en el arreglo ListaSubFolders[] definido por el programador, quedando la cantidad de los mismos en la variable entera del parámetro **NumeroDeFolders** transmitida por referencia.

**RETORNO:**

1) Retorna **null** si se presenta algún error en la apertura del fichero.

2) Retorna  la lista de folders que pertenecen al directorio seleccionado, y en el parámetro **NumeroDeFolders** la cantidad de los mismos.

**Método GetDirectoryFoldersPath**

**FORMATO:**

public string[] GetDirectoryFoldersPath(string Path, ref int NumeroDeSubDirectorios)

**DESCRIPCION:**

Este método obtiene la lista de **subfolders** contenidos dentro del directorio escogido por el usuario en **Path,** retornando el resultado en un arreglo de tipo **string**[] y depositándo también el número de subfolders de dicha lista en la variable del segundo parámetro
ref int NumeroDeSubDirectorios

(transmisión por referencia por lo que ref es necesario).

**PARAMETROS DE ENTRADA:**

string Path

Describe el Path del directorio que se desea listar.

ref int NumeroDeSubDirectorios

Aquí queda el número de subdirectorios del directorio seleccionado en Path

**Ejemplo:**

```
. . .
int NumeroDeFolders = 0;
string[] ListaSubFolders = null;
. . .
ListaSubFolders = GetDirectoryFoldersPath(@"D:\Empleados\Dep\Dep1.txt",
                                                    ref NumeroDeFolders);
. . .
```

*Nota:
En este ejemplo la lista de subfolders obtenida se copia en el arreglo ListaSubFolders[] definido por el programador, quedando la cantidad de los mismos en la variable entera del segundo parámetro **NumeroDeFolders**.

**RETORNO:**

1) Retorna **null** si se presenta algún error en la apertura del fichero.

2) Retorna  la lista de folders que pertenecen al directorio seleccionado y en el segundo parámetro la cantidad de los mismos**.**

**Método CreateFileParameter**

**FORMATO:**

public bool CreateFileParameter(string PathFile, string Parameter)

**DESCRIPCION:**

Este método escribe la línea de parámetros contenida en **Parameter** dentro del fichero descrito en **PathFile** que será creado.

**PARAMETROS DE ENTRADA:**

**1)** string PathFile
Contiene el Path del fichero de parámetros que será creado.

**2)** string Parameter
Contiene la línea de parámetros que será escrita en el fichero dado.

**Ejemplo:**

```
. . .
CreateFileParameter(@"C:\Param.txt", "ID=1020, Name=Rose, Age=25");
. . .
```

**RETORNO:**

1) Retorna **false** si se presenta algún error en la apertura del fichero.

2) Retorna **true** si todo fué OK.

**Método CreateFileParameterArray**

**FORMATO:**

public bool CreateFileParameterArray(string PathFile, string[] Parameter)

**DESCRIPCION:**
Este método escribe las líneas de parámetros contenidas en el array **Parameter** del segundo operando en el fichero descrito en **PathFile** que será creado.

**PARAMETROS DE ENTRADA:**

**1)** string PathFile
Describe el Path del fichero de parámetros que será creado.

**2)** string[] Parameter
Array que contiene las líneas de parámetros que serán escritas en el fichero dado.

**Ejemplo:**

```
. . .
string[] Lineas = {"Parametro1","Parametro2","Parametro3"};
CreateFileParameterArray(@"D:\Param.txt", Lineas);
. . .
```

En este ejemplo se crea el fichero **C:\Param.txt** y se escriben los artículos:
Parametro1
Parametro2
Parametro3

**RETORNO:**

1) Retorna **false** si se presenta algún error en la apertura del fichero.

2) Retorna **true** si todo fué OK.
Nota:
En Windows 10 el programador tiene que tener habilitado los derechos de escritura en la raiz del disco C:\ si desea crear su fichero parámetro en este lugar, pues usualmente el C:\ es el disco sistema y hay un mayor nivel de protección por parte del sistema. Por esta razón aconsejamos que utilice cualquier otro disco para este fin.

**Método StorageSize**

**FORMATO:**

public string StorageSize(double Unidad)

**DESCRIPCION:**

Este método analiza la **Unidad** de almacenamiento expresada en **bytes**, y retorna una subcadena expresada en la unidad más conveniente de almacenamiento equivalente a **Unidad**.

**PARAMETROS DE ENTRADA:**

1) double Unidad

Contiene el tamaño del fichero expresado en bytes.

**RETORNO:**

Unidad >= 0 & Unidad < 1000
retorna Equivalente + " Bytes" --> Ejemplo "567 Bytes"

Unidad >= 1000 & Unidad < 1000000
retorna Equivalente + " Kilobytes" --> Ejemplo "567 Kb"

Unidad >= 1000000 & Unidad < 1000000000
retorna Equivalente + " Megabytes" --> Ejemplo "567 Mb"

Unidad >= 1000000000 & Unidad < 1000000000000)
retorna Equivalente + " Gigabytes" --> Ejemplo "567 Gb"

Unidad >= 1000000000000 & Unidad < 1000000000000000
retorna Equivalente + " Terabytes" --> Ejemplo "567 Tb"
**Ejemplo:**
. . .
double Unidad = 800000;
string  Resultado = "";
Resultado =StorageSize(Unidad);
* Entonces en Resultado quedaría: 800000/1000  =  "800 Kb"